

Operaciones de Aprendizaje Automático III

MSc. Fernando Silva

fernandosilva.clases@gmail.com



Operaciones de Aprendizaje Automático III

Introducción a LLMOps y Ecosistema Open Source

MSc. Fernando Silva



Presentación



Fernando Silva

fernandosilva.clases@gmail.com



Apertura



¿En qué te especializas?

¿Qué esperas aprender en esta materia?

¿Qué recuerdas de la materia de PLN?

Organización

Descripción general

Este curso práctico se enfoca en LLMOps, orientado al desarrollo, despliegue, evaluación, monitoreo y optimización de aplicaciones basadas en modelos de lenguaje, utilizando principalmente herramientas open source. Se trabajará con fine-tuning, APIs, RAG, inferencia eficiente, observabilidad, seguridad y sistemas agénticos, culminando con la implementación de un sistema LLM completo y evaluable.

Distribución de notas

La calificación final se calculará como el promedio de las siguientes actividades:

- Tarea 1
- Tarea 2
- Tarea 3

Modalidad de trabajo: Las tareas podrán realizarse de manera individual o en grupos de hasta 3 integrantes.

Material

Todo el contenido de la materia estará disponible en un repositorio de GitHub, donde se publicarán los materiales. Las tareas deberán ser entregadas a través de Google Forms.

Organización

Sílabo

Clase	Temario
1	Introducción a LLMOps y Ecosistema Open Source
2	Entornos Reproducibles y Gestión de Modelos
3	Fine-Tuning de Small Language Models
4	APIs y Serving de Modelos Generativos
5	Optimización e Inferencia Eficiente de LLMs
6	Pipelines de Aplicaciones LLM y Automatización
7	Monitoreo, Evaluación y Observabilidad
8	Sistemas LLM en Producción y Tendencias Modernas

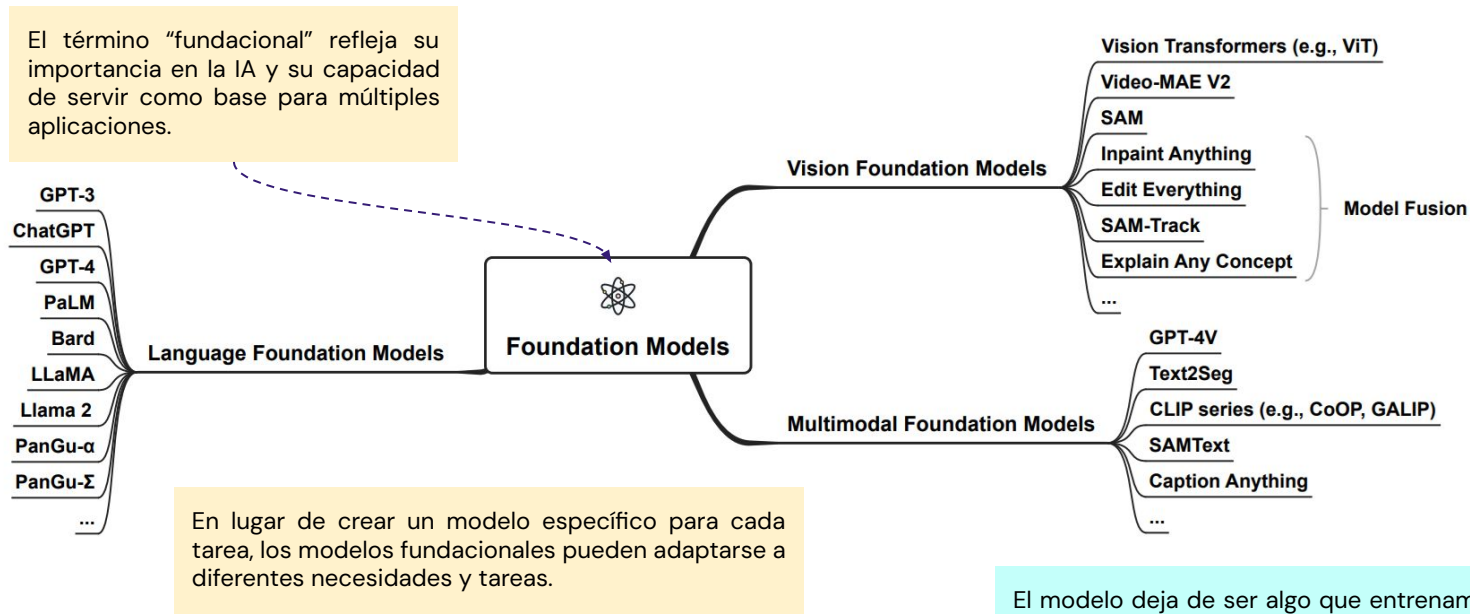
Agenda

- Ciclo MLOps Clásico y LLMs
- Ciclo de Vida de una Aplicación LLM
- Prompting, RAG y Fine-Tuning
- Ecosistema Open Source
- Tokenizers

Ciclo MLOps Clásico y LLMs

Foundation Models

Los modelos fundacionales (Foundation Models) surgieron a partir de los grandes modelos de lenguaje (LLM), que a su vez se originaron como simples modelos de lenguaje.



Foundation Models

ML Tradicional

Training

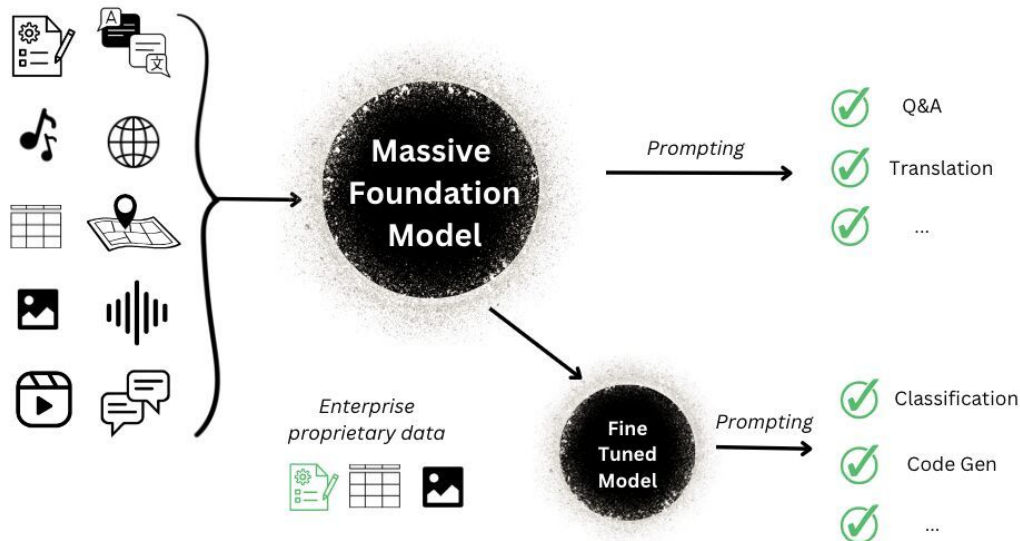
Tasks



- Modelos individuales aislados
- Requieren entrenamiento específico para cada tarea.
- Requieren mucho entrenamiento supervisado por humanos.

Foundation Model

Massive external data



- Modelo masivo multitarea
- Adaptable con poco o ningún entrenamiento
- Preentrenado mediante aprendizaje no supervisado

De Foundation Models a la Ingeniería de IA

- La ingeniería de IA se refiere al proceso de crear aplicaciones basadas en modelos fundacionales.
- La disponibilidad y accesibilidad de modelos fundacionales potentes generan cuatro factores que crean las condiciones ideales para el rápido crecimiento de la ingeniería de IA como disciplina:
 - **Factor 1: Capacidades de IA de propósito general**
Los modelos fundacionales no solo realizan mejor las tareas existentes, sino que permiten nuevas tareas y aplicaciones antes consideradas imposibles.
 - **Factor 2: Aumento de la inversión en IA**
El éxito de ChatGPT impulsó fuertemente la inversión en IA por parte de empresas y fondos de inversión.
 - **Factor 3: Baja barrera de entrada para crear aplicaciones de IA**
Las APIs de modelos permiten acceder fácilmente a modelos potentes sin tener que alojarlos ni administrar su infraestructura.
 - **Factor 4: La barrera se movió de entrenar a operar**
Construir un demo es hoy trivial. Lo difícil pasó a ser lo que viene después, evaluar sin métrica única, controlar el costo por request, versionar prompts, detectar regresiones cuando el proveedor cambia el modelo.

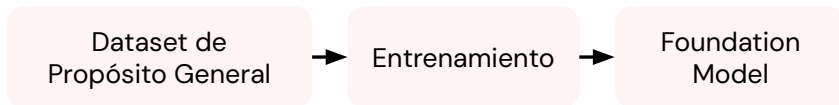
MLOps vs LLMOps

MLOps (Machine Learning Operations) es el conjunto de prácticas, herramientas y procesos para llevar modelos de Machine Learning desde el desarrollo hasta producción y mantenerlos funcionando de forma confiable.

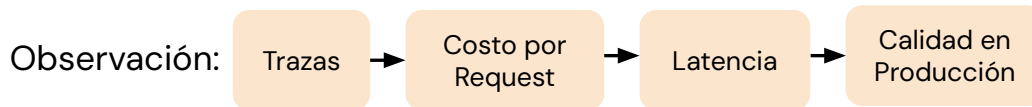
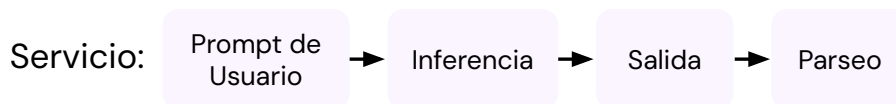
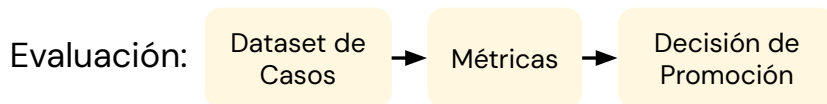
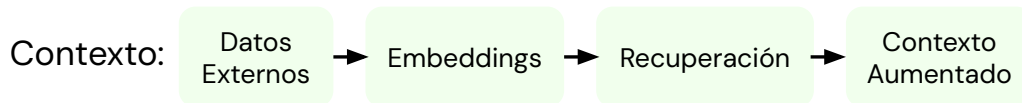
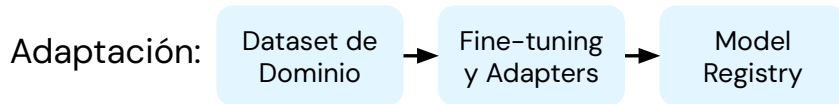
LLMOps (Large Language Model Operations) es la aplicación de principios de MLOps específicamente a LLMs y aplicaciones basadas en LLMs

MLOps vs LLMOps

Fuera de nuestro control:



Dentro de nuestro control:

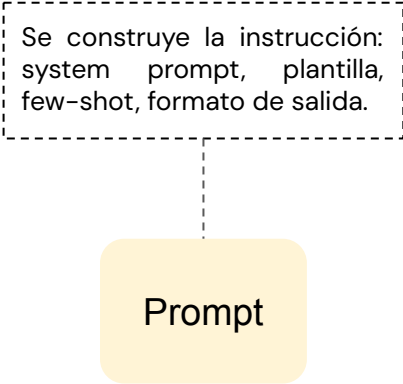


Ciclo de Vida de una Aplicación LLM

Ciclo de Vida de una Aplicación LLM

Se construye la instrucción:
system prompt, plantilla,
few-shot, formato de salida.

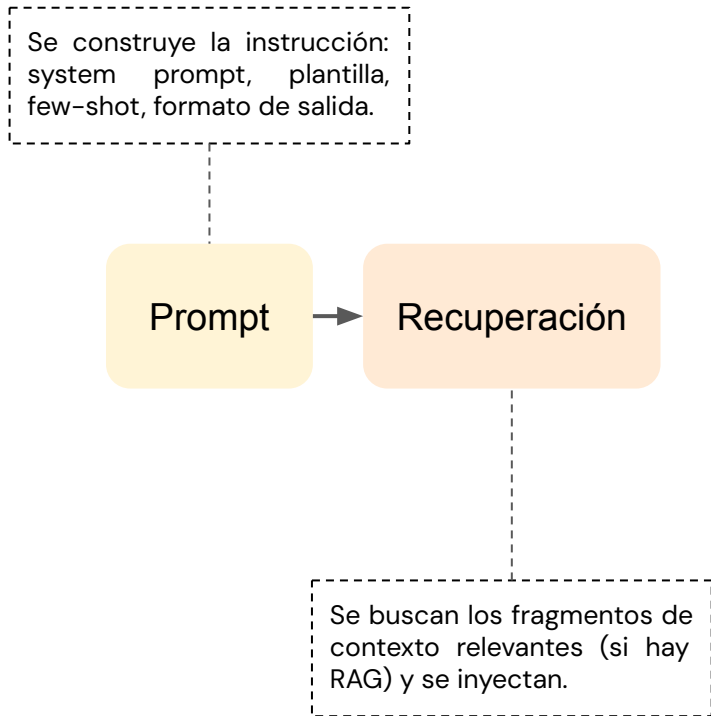
Prompt



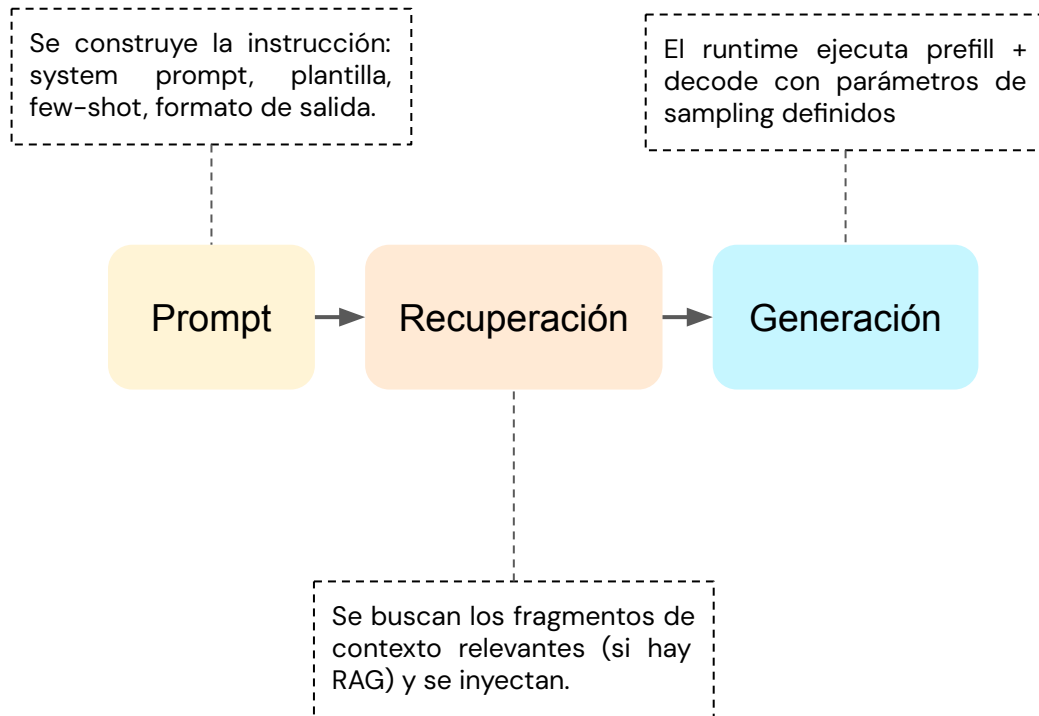
```
graph TD; A[Se construye la instrucción:  
system prompt, plantilla,  
few-shot, formato de salida.] -.- B[Prompt];
```

The diagram illustrates the initial step of the LLM application lifecycle. It features a dashed rectangular box at the top containing the text 'Se construye la instrucción: system prompt, plantilla, few-shot, formato de salida.' (The instruction is constructed: system prompt, template, few-shot, output format). A vertical dashed line connects this box to a solid yellow rounded rectangular box below it labeled 'Prompt'.

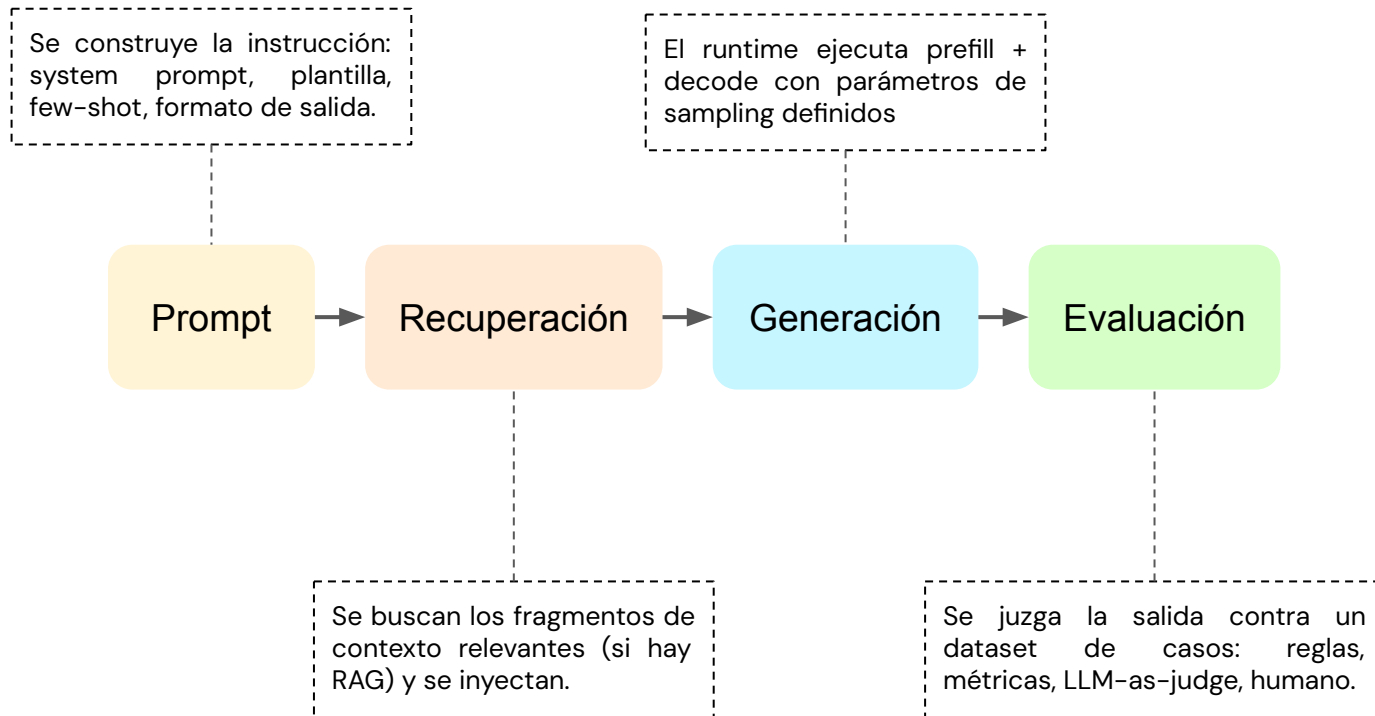
Ciclo de Vida de una Aplicación LLM



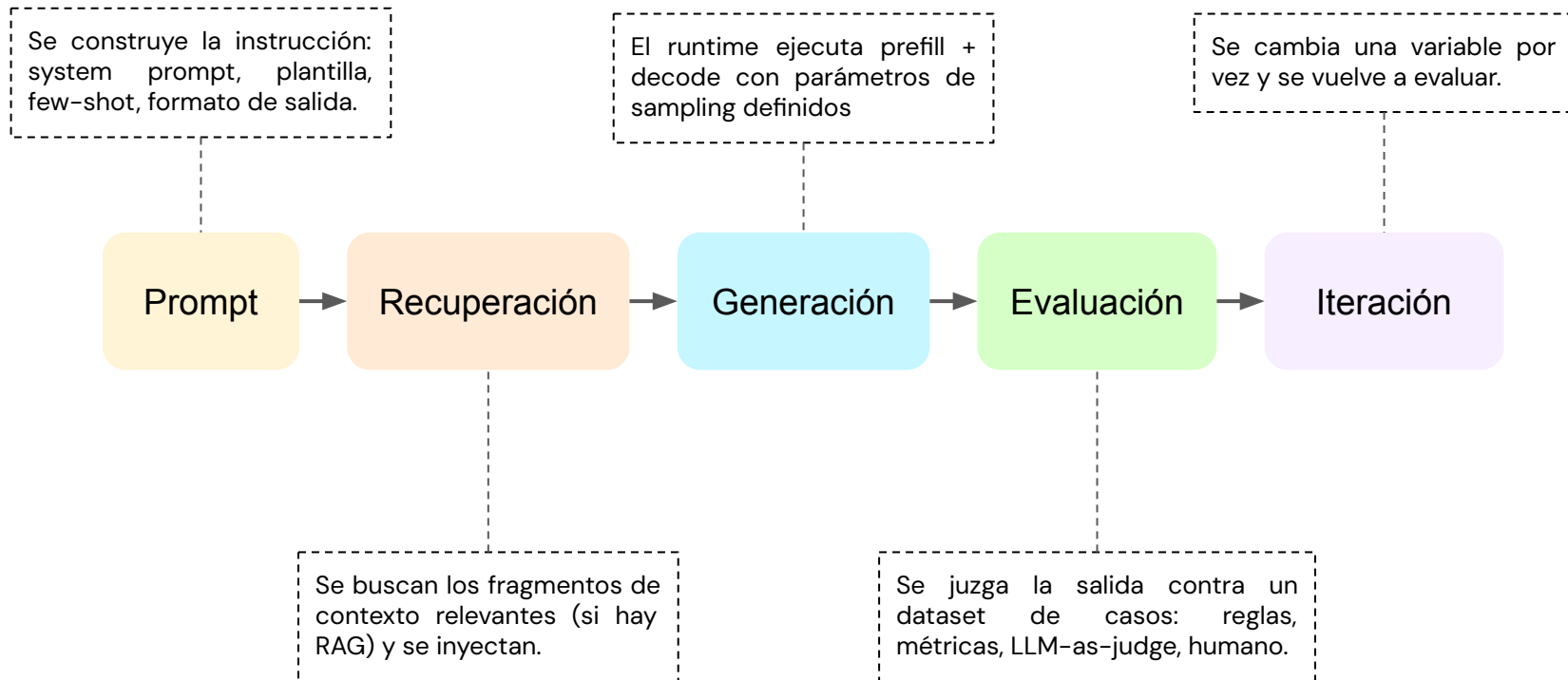
Ciclo de Vida de una Aplicación LLM



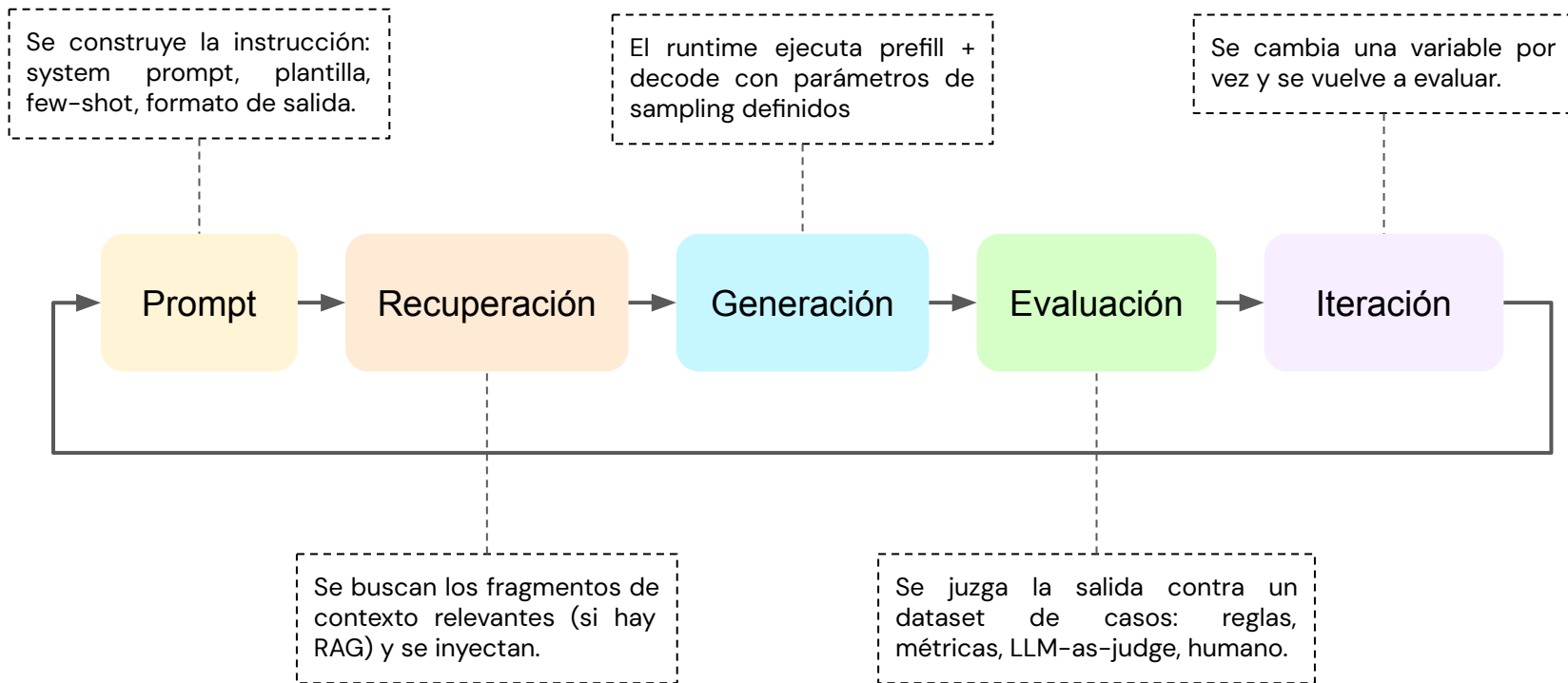
Ciclo de Vida de una Aplicación LLM



Ciclo de Vida de una Aplicación LLM



Ciclo de Vida de una Aplicación LLM



Diferencia clave con MLOps: el loop de iteración dura minutos, no días. No hay reentrenamiento en el medio. Eso hace que el cuello de botella se mueva del cómputo a la evaluación.

Prompting, RAG y Fine-Tuning

Prompting, RAG y Fine-Tuning

Prompting

Prompting

- **Qué cambia:** solo el texto de entrada, el modelo queda intacto.
- **Cuándo alcanza:** la información ya está en el modelo y el problema es de formato, tono, rol o estructura de razonamiento.
- **Costo:** horas, sin GPU.
- **Reversibilidad:** total, es un cambio de string, revertible en un commit.
- **Techo:** no agrega conocimiento que el modelo no tiene, y no arregla desconocimiento de datos privados o posteriores al corte de entrenamiento.
- **Riesgo típico:** prompts que crecen sin control, costo por request que sube en silencio.

Prompting, RAG y Fine-Tuning

RAG

Prompting

RAG

- **Qué cambia:** se inyecta conocimiento externo en el contexto, en tiempo de request.
- **El modelo sigue intacto.**
- **Cuándo corresponde:** el problema es de conocimiento, datos privados, cambiantes, o que requieren cita de fuente.
- **Costo:** días a semanas, infraestructura nueva: ingesta, chunking, embeddings, índice vectorial.
- **Reversibilidad:** alta, se actualiza el índice, no el modelo.
- **Ventaja decisiva:** trazabilidad. Podemos responder "¿de dónde salió esto?".
- **Riesgo típico:** la calidad pasa a depender del retriever. Un mal recall se ve como alucinación del modelo y se diagnostica mal.

Prompting, RAG y Fine-Tuning

Fine-tuning

Prompting

RAG

Fine-tuning

- **Qué cambia:** los pesos (o un adapter LoRA sobre ellos).
- **Cuándo corresponde:** el problema es de estructura, formato muy específico, estilo, jerga de dominio, tarea acotada donde un modelo chico debe igualar a uno grande.
- **Costo:** semanas. Dataset curado + GPU + evaluación + un artefacto más para versionar.
- **Reversibilidad:** baja, cada cambio del dataset exige reentrenar y reevaluar.
- **Beneficio secundario:** permite bajar de tamaño de modelo, menos costo de inferencia.
- **Riesgo típico:** se usa para meter conocimiento. Olvido catastrófico.

Prompting, RAG y Fine-Tun

Fácil y barato



Difícil y costoso

Prompting

RAG

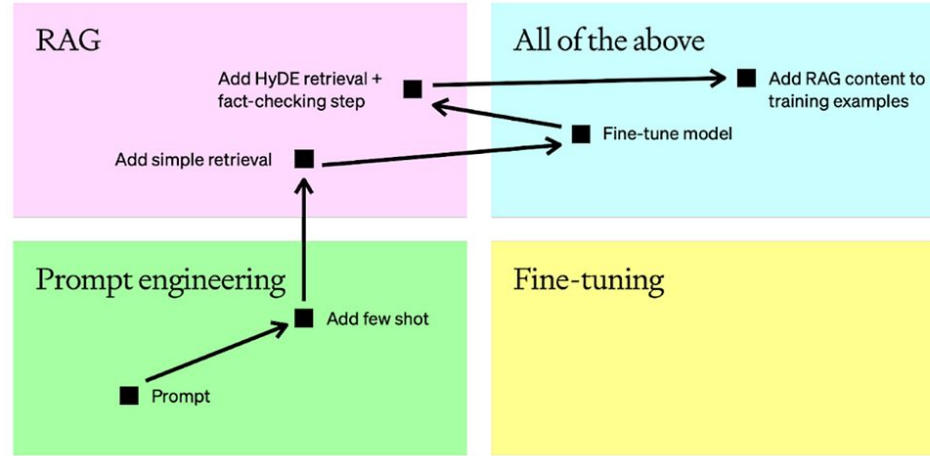
Fine-tuning

Cuadrante OpenAI

Optimización de contexto:
Qué necesita saber el modelo. Se ataca con RAG.

Context optimization

What the model needs to know



Optimización del modelo:
Cómo necesita comportarse. Se ataca con fine-tuning.

LLM optimization
How the model needs to act

Diagnóstico antes de elegir herramienta:

- **Falla porque no sabe algo:** eje vertical, RAG
- **Falla porque no responde como queremos:** eje horizontal, prompting, luego fine-tuning
- **Falla en ambos:** las dos cosas, en ese orden

Anti-patrones

1. Fine-tuning con la documentación de la empresa

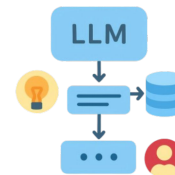
Situación: se necesita que el asistente conozca las políticas internas. Se hace fine-tuning sobre 3.000 páginas de documentación.



Resultado: el modelo adopta el registro corporativo pero sigue inventando plazos y números. Cada actualización de política exige reentrenar. No puede citar la fuente. Imposible auditar de dónde salió una respuesta.



Diagnóstico: problema de conocimiento tratado como problema de comportamiento. Correspondía RAG.



Anti-patrones

2. Metemos todo el manual en el contexto

Situación: la ventana de contexto es grande, así que se inyecta el documento completo en cada request en vez de construir un índice.



Resultado: el costo por request se multiplica, **la latencia (TTFT) crece con el cuadrado del contexto**, y la calidad empeora porque el modelo tiene dificultad para localizar información en el medio de contextos largos.



Diagnóstico: contexto largo no sustituye recuperación selectiva.



Prompting y RAG

[LINK](#)



Ecosistema Open Source

Ecosistema Open Source

Distribución

Alojar pesos, datasets,
model cards

Hugging Face Hub



Librerías base

Cargar y ejecutar modelos

*transformers, tokenizers,
datasets*



Ajuste eficiente

Fine-tuning con recursos
limitados

*peft, trl, accelerate,
bitsandbytes*



Runtime local / desarrollo

Inferencia simple en la
máquina propia

Ollama, llama.cpp



Ollama



llama.cpp

Serving de producción

Throughput, concurrencia,
batching

vLLM, TGI, SGLang



Se encarga de transformar un modelo
en un servicio escalable.

Cuantización

Reducir memoria y acelerar

*GGUF, AWQ, GPTQ,
bitsandbytes*



Recuperación

Embeddings e índices
vectoriales

*sentence-transformers,
Chroma, Qdrant*



Orquestación de aplicaciones

Encadenar pasos,
herramientas

LangChain, LlamaIndex



Tipos de Open Source

Categoría	Se publica	Se puede auditar	Licencia típica
Open source estricto	Pesos, código y datos	Todo	Apache 2.0, MIT
Open weights	Solo los pesos	El comportamiento, no el origen	Apache 2.0, MIT
Open weights restringido	Pesos con condiciones de uso	El comportamiento	Licencia comunitaria propia
Cerrado	Nada	Nada	Términos de servicio

La gran mayoría de los modelos que se llaman "abiertos" pertenecen a la segunda o tercera categoría.

Licencias

Cláusulas que aparecen con frecuencia en licencias comunitarias y que pueden invalidar un despliegue:

- **Umbral de usuarios activos por encima del cual se requiere una licencia comercial separada.**
- **Restricción sobre destilación:** prohibición de usar las salidas del modelo para entrenar otros modelos.
- **Requisito de atribución visible en el producto.**
- **Política de uso aceptable anexa, que forma parte vinculante de la licencia.**
- **Modelos gated:** exigen aceptación explícita en el Hub y un token de acceso, esto rompe pipelines si no se prevé.

Licencias

Cláusulas que aparecen con frecuencia en licencias comunitarias y que pueden invalidar un despliegue:

- **Umbral de usuarios activos por encima del cual se requiere una licencia comercial separada.**
- **Restricción sobre destilación:** prohibición de usar las salidas del modelo para entrenar otros modelos.
- **Requisito de atribución visible en el producto.**
- **Política de uso aceptable anexa, que forma parte vinculante de la licencia.**
- **Modelos gated:** exigen aceptación explícita en el Hub y un token de acceso, esto rompe pipelines si no se prevé.

Licencias

Las cláusulas anti-destilación existen porque las salidas de un modelo son un dataset de entrenamiento de alta calidad para otro modelo.

Forbes

INNOVATION > AI

Distillation: The New U.S.–China AI Fight

By [Craig S. Smith](#), Contributor. © Craig S. Smith, Eye on AI host and former N...

[Follow Author](#)

Published Jun 25, 2026, 06:16pm EDT, Updated Jun 30, 2026, 04:05pm EDT

America's most powerful AI models are now considered strategic national-security assets that China is allegedly trying to steal through a sophisticated technique known as [distillation](#).

Most recently, ChatGPT maker OpenAI said it will restrict access to its GPT-5.6 model series to a small group of companies approved by the Trump administration.

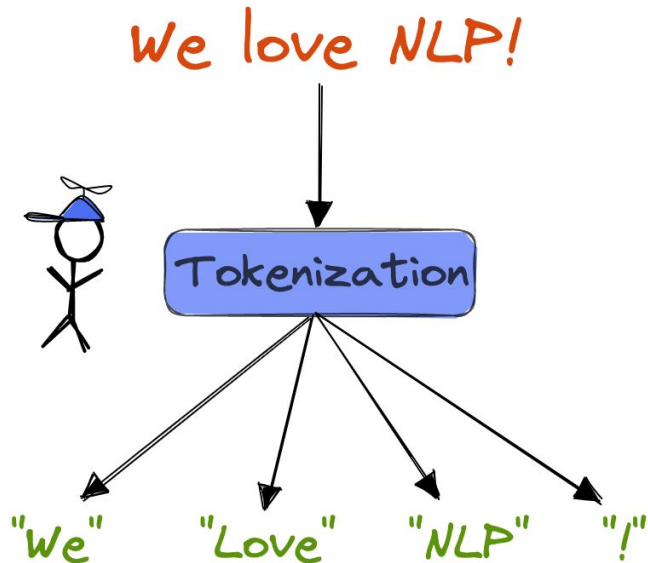
Meanwhile, on June 10, Anthropic sent a letter to Capitol Hill accusing one of China's premier AI labs, Alibaba's Qwen AI lab, of running what it called the largest known "distillation" attack on its Claude models to date.

Already, the U.S. government has slapped export controls on Anthropic's Mythos 5 and Fable 5 to prevent China or other foreign entities from getting access to the models. Mythos 5 and Fable 5 are considered so capable that they could be used to help hackers break into critical infrastructure — power grids, telecom networks, the rails behind global payments.

Tokenizers

¿Qué es un Tokenizer?

- Un tokenizer parte el texto en unidades (tokens) que el modelo puede mapear a vectores.
- No son palabras ni caracteres: son subpalabras aprendidas estadísticamente sobre un corpus de entrenamiento.



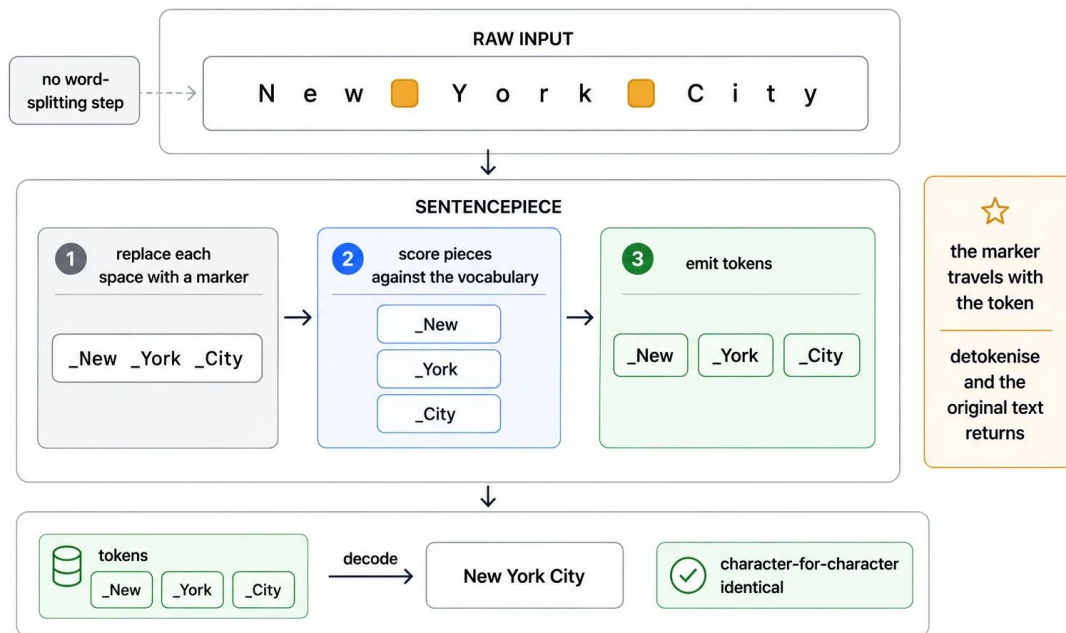
- El tokenizer es parte del modelo, no un preprocesamiento intercambiable.
- Es importante no cambiar de tokenizer sin cambiar de modelo.

Tokenizers

SentencePiece

- Trabaja sobre texto crudo sin pre-tokenizar por espacios, útil para idiomas sin separación clara de palabras

- SentencePiece tokeniza texto crudo y marca los espacios con `_`, así puede reconstruir el texto exacto al revertir la tokenización.
- Si eso falla, el output se corrompe en detalles pequeños (espacios, acentos, comillas).



Tokenizers

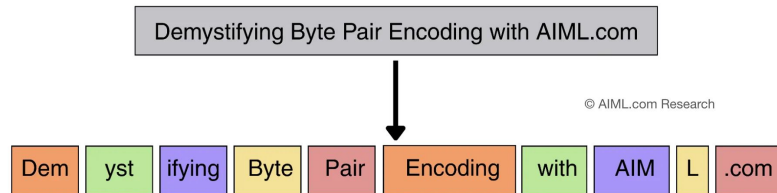
BPE (Byte-Pair Encoding)

- Fusiona iterativamente los pares de caracteres/bytes más frecuentes.

Algoritmo:

- 1. Vocabulario inicial:** todos los caracteres (o bytes) del corpus.
- 2. Buscar el par de símbolos adyacentes más frecuente y fusionarlo en un símbolo nuevo.**
- 3. Repetir k veces.** El vocabulario final son las k fusiones más rentables del corpus.

Las secuencias frecuentes en el corpus de entrenamiento se vuelven un solo token, las raras se parten en muchos. **Por eso el español cuesta más tokens en un modelo entrenado mayormente en inglés.**



Tokenizers

WordPiece

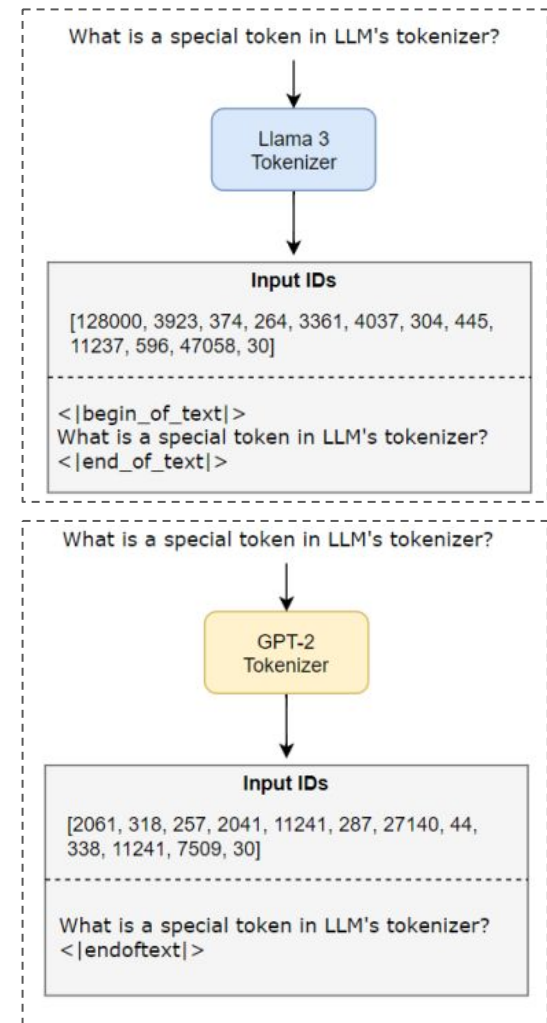
- WordPiece es el tokenizer de la familia BERT y derivados. Es decir: de la mayoría de los modelos de embeddings que se usan para RAG.
- Los generativos modernos usan casi siempre BPE a nivel byte o SentencePiece.

Entonces, el pipeline de RAG, entonces, tiene habitualmente dos tokenizers distintos: uno para indexar y otro para generar.



Tokenizers

- Cada modelo tiene su propio tokenizer, entrenado sobre su propio corpus.
- Un mismo texto en español puede producir más tokens en un modelo entrenado mayoritariamente en inglés que en uno con buena representación de español.
- Esto es importante, ya que afecta directamente costo, latencia y cuánto contenido real entra en la ventana de contexto.



Ventana de Contexto

La ventana de contexto es la cantidad máxima de tokens que el modelo puede procesar en una sola llamada.

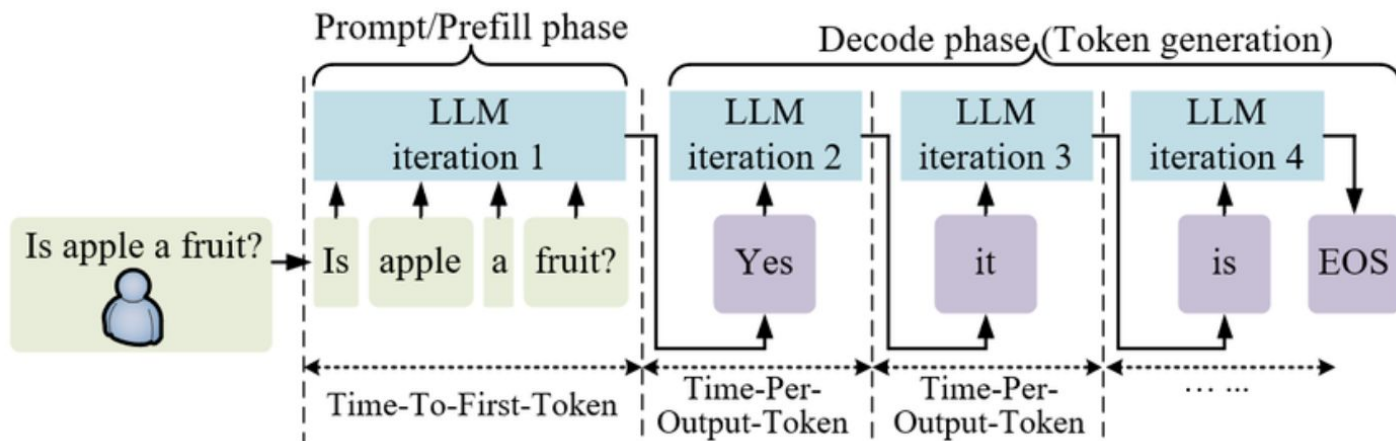
- **Costo cuadrático de atención:** el mecanismo de self-attention compara cada token con todos los demás, por lo que el cómputo (y en la práctica la latencia del prefill) crece con el cuadrado de la longitud del contexto. Duplicar el contexto no duplica el costo, lo multiplica más que proporcionalmente.
- **"Lost in the middle":** modelos con ventanas largas muestran peor recall sobre información ubicada en el medio del contexto que sobre lo que está al principio o al final.

	the	cat	sat	on	the	mat
the	-	low	low	low	-	low
cat	high	-	high	medium	low	medium
sat	low	high	-	high	low	medium
on	low	medium	high	-	low	high
the	-	low	low	low	-	low
mat	low	medium	medium	high	low	-

Inferencia en los LLMs

La inferencia consiste en utilizar un modelo de machine learning previamente entrenado para procesar datos nuevos que no ha visto durante su entrenamiento.

En el caso de los LLM, la inferencia ocurre cuando se recibe una entrada del usuario, como un prompt, y esta se procesa mediante los parámetros aprendidos por el modelo para generar una respuesta.



Inferencia en los LLMs

.Las cuatro métricas que vamos a monitorear toda la materia:

- **TTFT (Time To First Token):** Cuánto tarda en aparecer el primer token. Domina la percepción de "responde rápido". Depende del largo del prompt (prefill). Es lo que empeora cuando inflamamos el contexto.
- **TPOT (Time Per Output Token):** Cuánto tarda cada token siguiente. Define la sensación de fluidez del streaming. Depende del tamaño del modelo y del hardware, no del largo del prompt.
- **Latencia total = $TTFT + (TPOT \times \text{tokens generados})$**
- **Throughput:** tokens/segundo agregados en todo el servidor. Es la métrica de **costo**. Cuántos usuarios sostiene una GPU.

Trade off: En serving se quiere optimizar latencia (batches chicos, poca espera) y optimizar throughput (batches grandes, más espera) son objetivos opuestos.

Token como unidad de costo

- El token es la unidad atómica de tres cosas a la vez:

Dimensión	Cómo se relaciona con el token
Costo	Los proveedores cobran por token, con precio distinto para tokens de entrada y de salida (la salida suele ser más cara, porque cada token generado requiere un forward pass completo).
Latencia	Time to First Token (TTFT) depende del largo del prompt (prefill); el resto del tiempo de respuesta depende de cuántos tokens hay que generar, uno por uno.
Memoria	La memoria de inferencia crece linealmente con contexto × batch size.